Session-Aware Real-Time Recommendation over E-Commerce Clickstreams

ALS Fold-In with Kafka and Spark Structured Streaming on a Resource-Constrained Hadoop Cluster

Kirupashankar Chockkanathan, Padmacharan R, Prapanjan I, Shree Santh B

Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham

23AID302 Big Data Analytics – Final Review

Kirupashankar Chockkanathan	CB.AI.U4AID24123
Padmacharan R	CB.AI.U4AID24136
Prapanjan I	CB.AI.U4AID24140
Shree Santh B	CB.AI.U4AID24152

Faculty in charge: Dr. Sreeja

Problem statement, objectives, why big data

Problem. Recommend products that react to what a visitor clicks *right now*, even for anonymous visitors, at a data size no single machine handles comfortably.

Objectives

- Deploy HDFS (1 NameNode, 3 DataNodes), YARN, Spark, Kafka in 16 GB RAM
- Ingest JSON clicks + relational tables; clean; measure data quality
- ALS **fold-in** vs. co-occurrence vs. batch ALS, leakage-free temporal evaluation
- Kafka → Spark Streaming recommender with measured latency
- Measure Spark/HDFS tuning choices

Why big data (5 Vs)

- **Volume:** 109.95 M events, 26.3 GiB JSON (79 GiB with 3 replicas)
- **Velocity:** Kafka replay, 1,500 events/s (simulated rate)
- **Variety:** JSON + relational tables
- **Veracity:** 0.37% bad rows, bot-like sessions
- **Value:** next-item accuracy, engagement windows

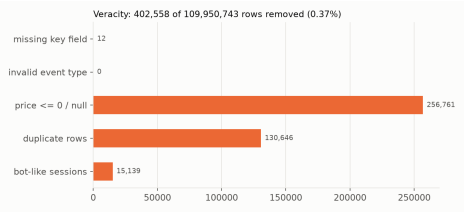
Base paper: Ludewig et al., UMUAI 2021 – simple neighbour methods often beat complex models. We test that claim in a **distributed streaming** setting.

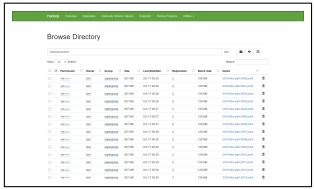
Dataset: source, size, volume / variety / velocity, attributes

Part	Variety	Content
Clickstream	semi-structured JSON Lines	REES46 store, Oct+Nov 2019, **109,950,743** events, 26.3 GiB, 106 files (real)
products, orders, order_items	PostgreSQL tables	206,876 / 1.40 M / 1.66 M rows (real-derived)
users, inventory	PostgreSQL tables	5.3 M / 207 k rows (**synthetic**)

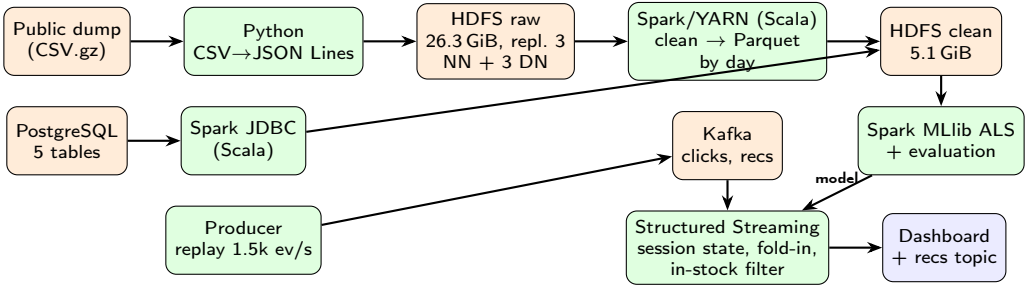
Attributes: event_time, event_type (view/cart/purchase/remove), product_id, category_code, brand, price, user_id, user_session.

Source: Kaggle `mkechinov/ecommerce-behavior-data-from-multi-category-store` (CSV download converted to JSON; CSV discarded – faculty JSON/CSV rule). Velocity is a replay of recorded events (simulated rate).



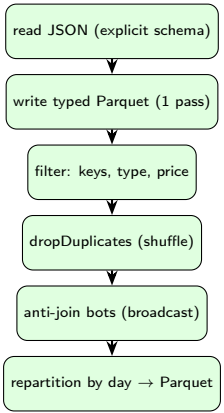


System architecture



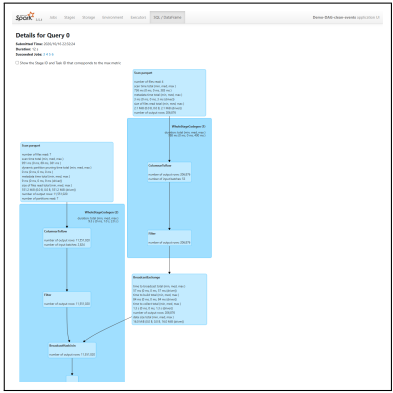
Docker on WSL2 (12 GB): 1 NameNode, 3 DataNodes, ResourceManager, 1 NodeManager, HistoryServer, Kafka (KRaft), PostgreSQL, Spark 3.5.1 client.

Spark workflow: DataFrame operations, DAG, partitioning, caching



Choices (measured, 1 run each)

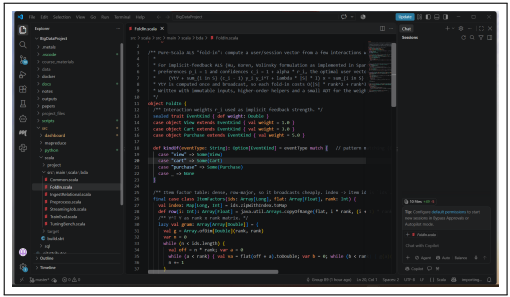
- Parquet scan 1.1 s vs JSON 8.8 s
- 8 files read 0.2 s vs 500 small files 1.5 s
- Broadcast join 1.1 s vs sort-merge 3.1 s
- Partition pruning 0.11 s vs 1.84 s
- 24 shuffle partitions 3.1 s; 400: 12.6 s
- Cache: 14.8 s with fill vs 5.6 s uncached; wins only on re-use (3.9 s)



Spark UI plan (week of clean events, broadcast join): only 7 day-partitions read

Scala implementation: key code and project structure

```
// FoldIn.scala: re-solve a session vector, no retraining
def sessionVector(f: ItemFactors, session: Seq[(Long, Double)],
    lambda: Double, alpha: Double): Option[Array[Double]] = {
  val known = session.flatMap { case (id, r) =>
    f.index.get(id).map(_ -> r) }
  if (known.isEmpty) None else {
    val a = Array.tabulate(f.rank, f.rank)((i, j) =>
      f.gram(i)(j) + (if (i == j) lambda * known.size else 0.0))
    val b = new Array[Double](f.rank)
    known.foreach { case (idx, w) => /* add (c-1) y y^T and c y */ }
    Some(solveSpd(a, b))            // Cholesky solve
  }
}
// pattern matching on event kinds (sealed trait)
def kindOf(t: String): Option[EventKind] = t match {
  case "view"     => Some(View)
  case "cart"     => Some(Cart)
  case "purchase" => Some(Purchase)
  case _          => None
}
```



VS Code: project tree and FoldIn.scala

src/scala/.../bda/: Common, Preprocess, IngestRelational, FoldIn, TrainEval, StreamingJob, TuningBench. Python: converter, producer, charts, dashboard. MapReduce: Streaming job (counts match Spark).

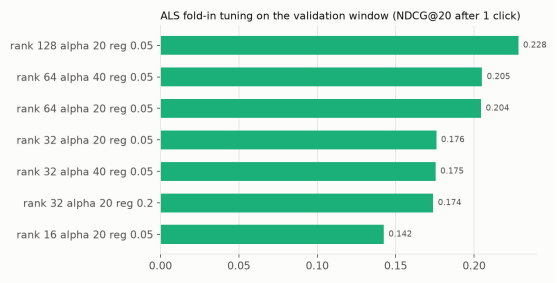
Spark ML pipeline: features, algorithms, tuning

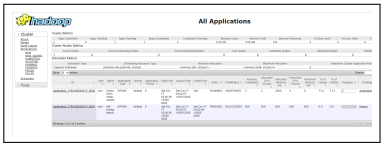
Features: implicit strength view=1, cart=3, purchase=5 summed per (user,item); catalogue = 45,863 items with $\geq$30 training users; 25% user sample, 10.96 M training pairs.

Algorithms: popularity; MLlib implicit ALS (batch); **ALS + fold-in**; item co-occurrence; hybrid (reciprocal-rank fusion).

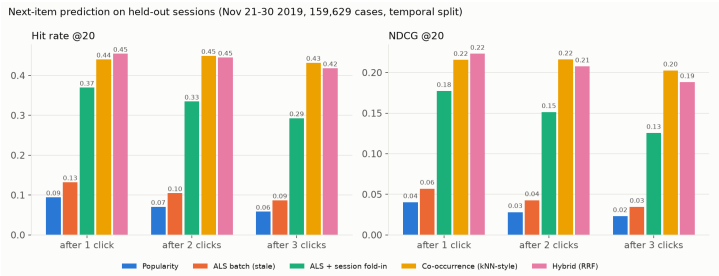
Protocol: strict temporal split (train to Nov 20, test Nov 21–30); 159,629 next-item cases; HR/MRR/NDCG@20.

Tuning on a validation window (never the test window): rank 16/32/64/128, alpha 20/40, reg 0.05/0.2. Rank 128: NDCG 0.228 vs 0.205 (rank 64) but 4.5$\times$ training time (304 s vs 67 s) $\rightarrow$ final rank 64.





Results: metrics, model comparison, business insight



After 1 click (HR@20 / NDCG@20)
Popularity 0.094 / 0.040
Batch ALS (stale) 0.132 / 0.057
ALS + fold-in 0.370 / 0.177
Co-occurrence 0.440 / 0.216
Hybrid 0.454 / 0.223

Insight: freshness matters (fold-in $\approx 2.8\times$ stale ALS), but a simple co-occurrence list is the strongest single model – consistent with the base paper.

Streaming: 44,613 recs from 135 k events; latency p50 8.5 s, p99 20.1 s.

Challenges, limitations, conclusion, future work

Challenges and limitations

- 16 GB laptop: WSL2 12 GB, 1.25 GB executors, 1 NodeManager
- 25% user sample, single runs, no significance tests
- Catalogue limited to 45.9 k popular items
- Users and inventory are synthetic; velocity is simulated replay
- Latency is seconds, not milliseconds
- No neural / full SKNN baselines; fold-in weakens with longer sessions (cause not investigated)
- 3 DataNodes share one disk (no real fault tolerance)

Conclusion

A reproducible Hadoop/Spark/Kafka platform processes 110 M events on a laptop. Fold-in makes ALS fresh at ms cost; co-occurrence still wins; hybrid helps at the first click.

Future work

- Prequential evaluation with significance tests
- GRU4Rec / SKNN baselines (Kaggle GPU)
- Multi-broker Kafka, Kafka Connect to HDFS
- Online A/B test

References and GitHub link I

GitHub: `https://github.com/kirupashankarchockkanathan/Clickstream-Recommender`

M. Ludewig, N. Mauro, S. Latifi, D. Jannach, "Empirical analysis of session-based recommendation algorithms," *User Model. User-Adap. Inter.*, vol. 31, pp. 149–181, 2021, doi:10.1007/s11257-020-09277-1.

S. Wang, L. Cao, Y. Wang, Q. Z. Sheng, M. A. Orgun, D. Lian, "A survey on session-based recommender systems," *ACM Comput. Surv.*, vol. 54, no. 7, art. 154, 2021, doi:10.1145/3465401.

M. Jakomin, Z. Bosnić, T. Curk, "Simultaneous incremental matrix factorization for streaming recommender systems," *Expert Syst. Appl.*, vol. 160, art. 113685, 2020, doi:10.1016/j.eswa.2020.113685.

J. Vinagre, A. M. Jorge, C. Rocha, J. Gama, "Statistically robust evaluation of stream-based recommender systems," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 7, 2021, doi:10.1109/TKDE.2019.2960216.

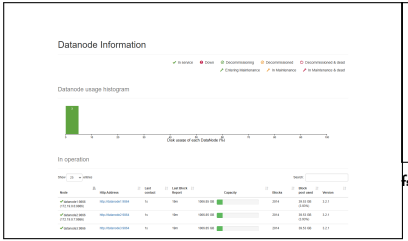
Y. Ji, A. Sun, J. Zhang, C. Li, "A critical study on data leakage in recommender system offline evaluation," *ACM Trans. Inf. Syst.*, vol. 41, no. 3, art. 75, 2023, doi:10.1145/3569930.

T. P. Raptis, A. Passarella, "A survey on networked data streaming with Apache Kafka," *IEEE Access*, 2023, doi:10.1109/ACCESS.2023.3303810.

M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, pp. 56–65, 2016, doi:10.1145/2934664.

M. Kechinov, "eCommerce behavior data from multi-category store," REES46 Open CDP, Kaggle.

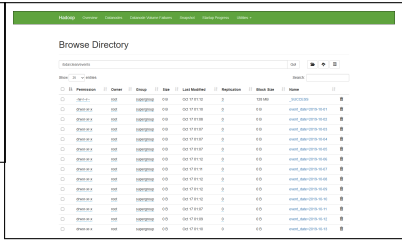
Backup: HDFS evidence (replication 3, three live DataNodes)



DataNodes tab: 3 in service

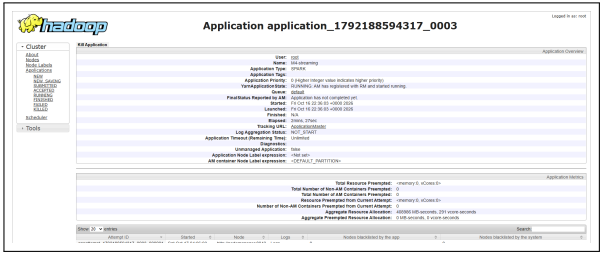


fsck: HEALTHY, avg. replication 3.0



clean data, one folder per day

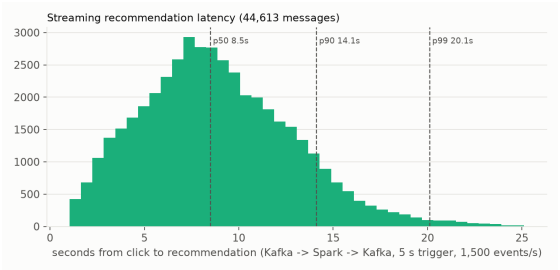
Backup: streaming demo evidence



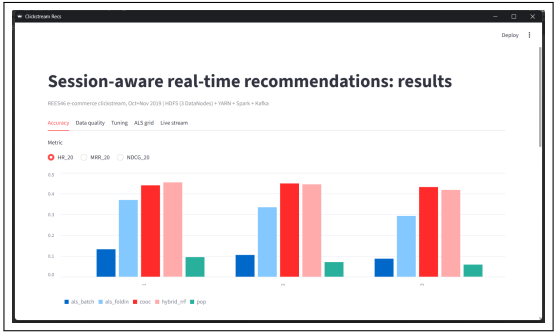
YARN: M4-streaming application RUNNING



messages on Kafka topic recs (session, clicks, 10 recommended items)



Backup: dashboard and tuning results



Streamlit dashboard, Accuracy tab (grouped bars)

